

Course Report

Assignment 2 - Task 1

Optimizer Performance

on Non-Convex Functions

**A from-scratch comparison of Gradient Descent, Momentum, Adam, RMSprop, and Adagrad
on the Rosenbrock and simplified 1-D Ackley functions**

1. Objective

This report evaluates five gradient-based optimization algorithms. Gradient Descent, SGD with Momentum, Adam, RMSprop, and Adagrad, on two classic non-convex test functions: the 2-D Rosenbrock function and a simplified 1-D Ackley function. All five optimizers were implemented from scratch in NumPy (no library optimizers were used). Each optimizer was run at three learning rates (0.01, 0.05, and 0.1), and results were recorded for convergence behaviour, iteration count, execution time, the final solution reached, and the final function value.

2. Methodology

2.1 Test Functions

Rosenbrock function (2-D): $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$, global minimum $f = 0$ at $(x, y) = (1, 1)$. Optimization begins from the classic starting point $(-1.2, 1.0)$, which sits inside the function's narrow, curved valley.

Simplified 1-D Ackley function: $f(x) = -20 \cdot \exp(-0.2 \cdot |x|) - \exp(\cos(2\pi x)) + 20 + e$, global minimum $f = 0$ at $x = 0$. The cosine term makes this function highly multimodal, with many shallow local minima surrounding the global one. Optimization begins from $x = 2.0$.

2.2 Optimizers and Experimental Setup

- Optimizers: Gradient Descent, SGD with Momentum ($\beta = 0.9$), Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$), RMSprop ($\beta = 0.9$, $\epsilon = 1e-8$), Adagrad ($\epsilon = 1e-8$).
- Learning rates tested: 0.01, 0.05, 0.1 — applied identically to every optimizer.
- Stopping rule: iteration stops when the step in parameter space falls below a tolerance of $1e-8$ ("converged"), when values become non-finite ("diverged"), or after 10,000 iterations ("max_iter").
- All 30 experiments (5 optimizers $\times$ 3 learning rates $\times$ 2 functions) were run with a fixed random seed (42) for reproducibility.

3. Results — Rosenbrock Function

Table 1 reports the outcome of every optimizer / learning-rate combination on the Rosenbrock function.

Optimizer	LR	Status	Iterations	Time (s)	Final f(x)	Final Solution
Gradient Descent	0.01	diverged	6	0.0007	inf	1.12e+78, 8.54e+51
Gradient Descent	0.05	diverged	5	0.0004	inf	6.52e+130, 2.20e+87
Gradient Descent	0.1	diverged	5	0.0007	inf	6.49e+169, 2.76e+113
Momentum	0.01	diverged	6	0.0003	inf	1.25e+90, 9.22e+59
Momentum	0.05	diverged	5	0.0003	inf	6.42e+130, 2.18e+87
Momentum	0.1	diverged	5	0.0005	inf	6.48e+169, 2.76e+113
Adam	0.01	converged	5,531	0.2415	3.10e-13	0.999999, 0.999999
Adam	0.05	converged	2,546	0.1595	1.39e-13	1.000000, 0.999999
Adam	0.1	converged	1,793	0.1168	7.70e-14	1.000000, 0.999999
RMSprop	0.01	max_iter	10,000	0.4302	2.22e-02	0.990144, 0.965534
RMSprop	0.05	max_iter	10,000	0.5174	4.36e-01	0.725000, 0.465625
RMSprop	0.1	max_iter	10,000	0.6336	1.05e+00	0.300000, 0.015000
Adagrad	0.01	max_iter	10,000	0.6532	3.57e+00	-0.889405, 0.797184
Adagrad	0.05	max_iter	10,000	0.7196	5.12e-02	0.773802, 0.598039
Adagrad	0.1	max_iter	10,000	0.7618	6.27e-04	0.974974, 0.950495

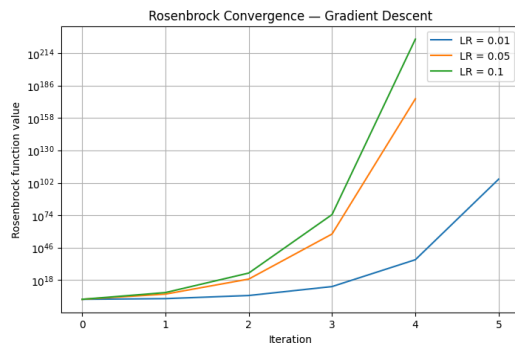
Table 1. Rosenbrock optimization results — final solution, function value, iterations, and time.

3.1 Observations

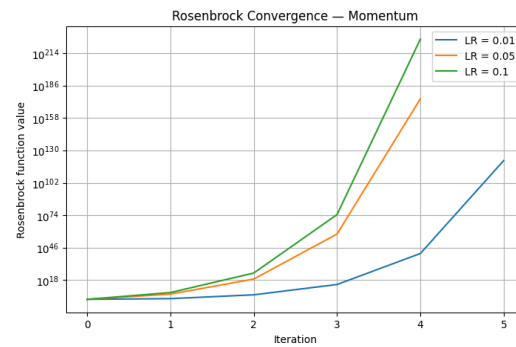
- Plain Gradient Descent and Momentum diverged at every tested learning rate (0.01, 0.05, 0.1). Rosenbrock's narrow curved valley produces very large gradients away from the optimum, and a fixed step size (even as small as 0.01) overshoots the valley wall, causing the iterate to blow up to numerical infinity within 5–6 steps.
- Adam was the only optimizer to fully converge on Rosenbrock at all three learning rates, reaching a solution within $1e-6$ of the true optimum (1, 1) with a final function value on the order of $1e-13$ to $1e-14$. Its adaptive, bias-corrected step sizing keeps updates well-scaled even in the steep parts of the valley.
- Higher learning rates helped Adam converge faster: 1,793 iterations at LR = 0.1 versus 5,531 iterations at LR = 0.01, while reaching an even lower final function value.
- RMSprop and Adagrad made steady progress but did not fully converge within the 10,000-iteration budget. Adagrad's accumulating denominator caused its effective step size to shrink over time; at LR = 0.1 it reached $f(x) \approx 6.3 \times 10^{-4}$, its best result, whereas at LR = 0.01 its step size shrank too quickly and it stalled far from the optimum ($f(x) \approx 3.57$).
- RMSprop's best Rosenbrock result was at the lowest learning rate (0.01, $f(x) \approx 0.022$); larger learning rates caused it to oscillate around the valley floor without settling.

3.2 Convergence Plots

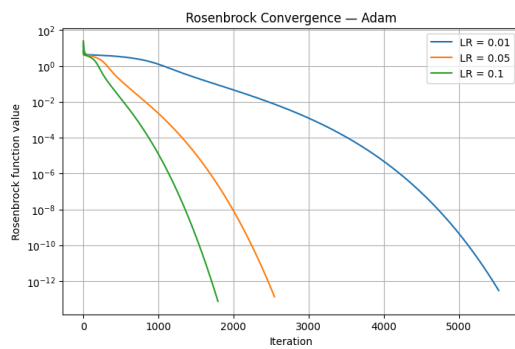
The following plots show function value versus iteration (log scale) for each optimizer at learning rates 0.01, 0.05, and 0.1.



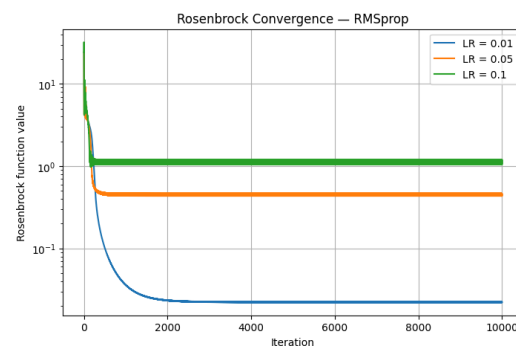
Gradient Descent — Rosenbrock



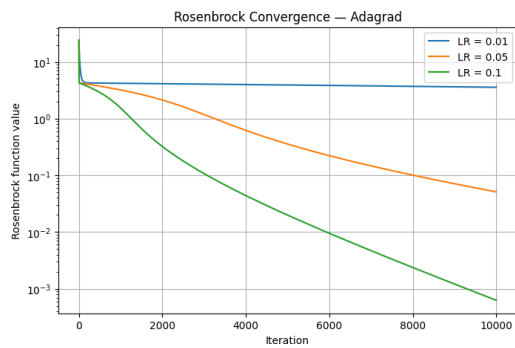
Momentum — Rosenbrock



Adam — Rosenbrock



RMSprop — Rosenbrock



Adagrad — Rosenbrock

4. Results — Ackley Function (1-D)

Table 2 reports the outcome of every optimizer / learning-rate combination on the simplified 1-D Ackley function.

Optimizer	LR	Status	Iterations	Time (s)	Final $f(x)$	Final Solution
Gradient Descent	0.01	converged	5	0.0003	6.5596	1.974452
Gradient Descent	0.05	max_iter	10,000	0.7594	3.2716	0.315579
Gradient Descent	0.1	max_iter	10,000	0.6860	3.5955	0.947334
Momentum	0.01	converged	252	0.0146	6.5596	1.974452
Momentum	0.05	max_iter	10,000	0.8656	20.2608	19.881060
Momentum	0.1	max_iter	10,000	0.6707	15.2342	5.334521
Adam	0.01	converged	238	0.0101	6.5596	1.974452
Adam	0.05	converged	247	0.0101	6.5596	1.974452
Adam	0.1	converged	274	0.0110	6.5596	1.974452
RMSprop	0.01	max_iter	10,000	0.3626	6.5609	1.979400
RMSprop	0.05	max_iter	10,000	0.3531	6.5887	1.998115
RMSprop	0.1	max_iter	10,000	0.3850	6.6613	2.018482
Adagrad	0.01	converged	40	0.0014	6.5596	1.974452
Adagrad	0.05	converged	18	0.0007	6.5596	1.974452
Adagrad	0.1	converged	29	0.0010	6.5596	1.974452

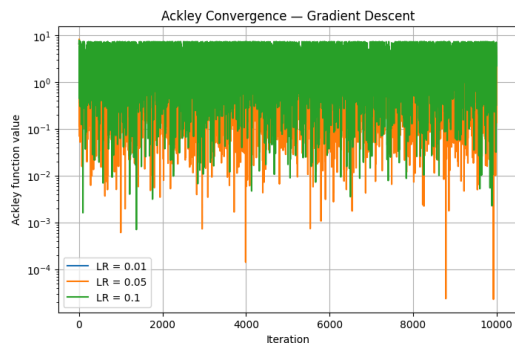
Table 2. Ackley optimization results — final solution, function value, iterations, and time.

4.1 Observations

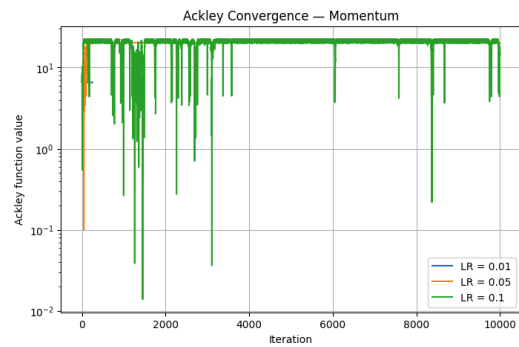
- No optimizer reached the true global minimum at $x = 0$. Starting from $x = 2.0$, the cosine ripple in the Ackley function creates a local basin near $x \approx 1.97$ ($f(x) \approx 6.56$), and most runs settled into it and reported "converged" — meaning the step size dropped below tolerance, not that the global optimum was found.
- Adam, Momentum (at $LR = 0.01$), and Adagrad all converged quickly to this same local minimum ($x \approx 1.974$), since their adaptive scaling shrinks the step as soon as it enters the local basin — Adagrad did so in as few as 18 iterations.
- Counter-intuitively, the two runs that came closest to the true global optimum were plain Gradient Descent at $LR = 0.05$ ($x \approx 0.316$, $f(x) \approx 3.27$) and $LR = 0.1$ ($x \approx 0.947$, $f(x) \approx 3.60$) — both of which hit the 10,000-iteration cap rather than "converging". Their larger, non-adaptive steps kept them oscillating across the ripples for longer instead of settling immediately into the first local minimum encountered, which let them drift closer to $x = 0$.
- Momentum became unstable at higher learning rates on this function: at $LR = 0.05$ it overshoot out to $x \approx 19.9$ ($f(x) \approx 20.3$), far worse than its $LR = 0.01$ result — the accumulated velocity carried it past the region of interest.
- RMSprop stayed near the same local minimum ($x \approx 1.98$ – 2.02) at all three learning rates without fully converging, consistently landing slightly worse than the local-minimum value of 6.5596.

4.2 Convergence Plots

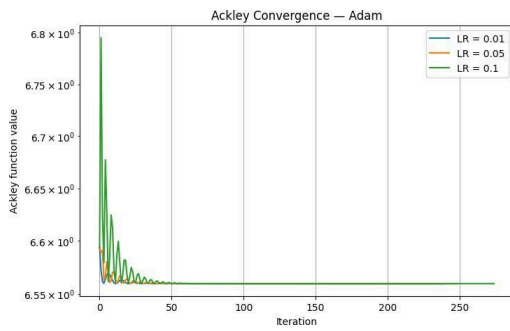
The following plots show function value versus iteration (log scale) for each optimizer at learning rates 0.01, 0.05, and 0.1.



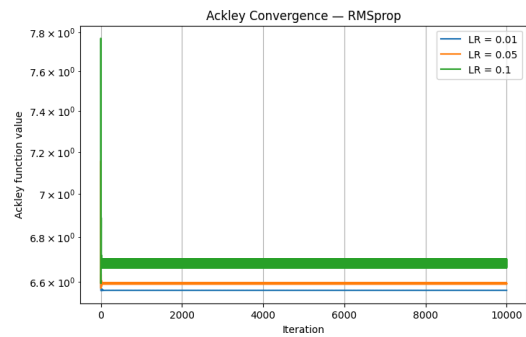
Gradient Descent — Ackley



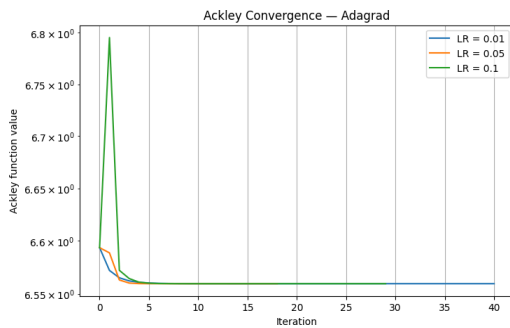
Momentum — Ackley



Adam — Ackley



RMSprop — Ackley



Adagrad — Ackley

5. Optimizer Comparison at a Fixed Learning Rate

To compare optimizers directly (independent of learning-rate tuning), the plots below overlay all five optimizers on the same axes at a fixed learning rate of 0.01.

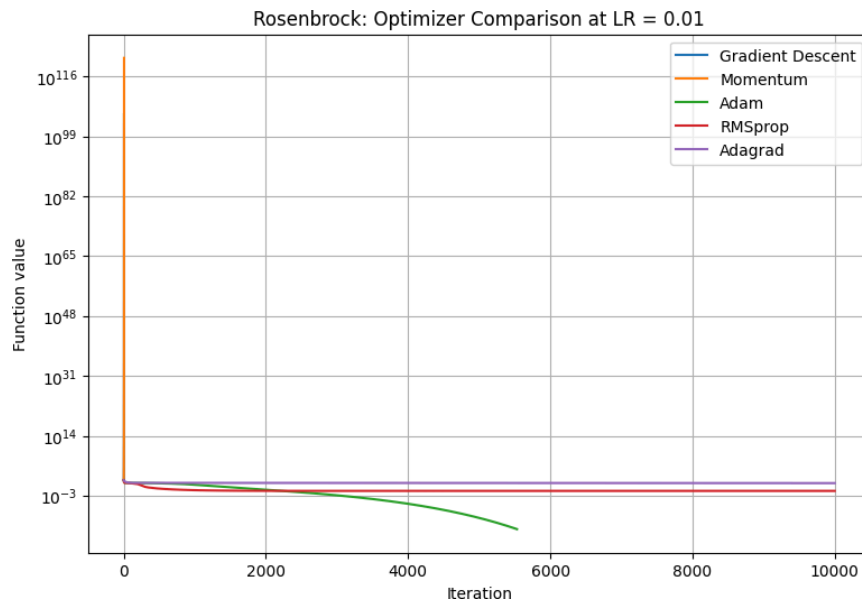


Figure A. Rosenbrock: all optimizers at LR = 0.01. Adam separates clearly from the rest, continuing to descend for thousands of iterations while Gradient Descent and Momentum diverge almost immediately.

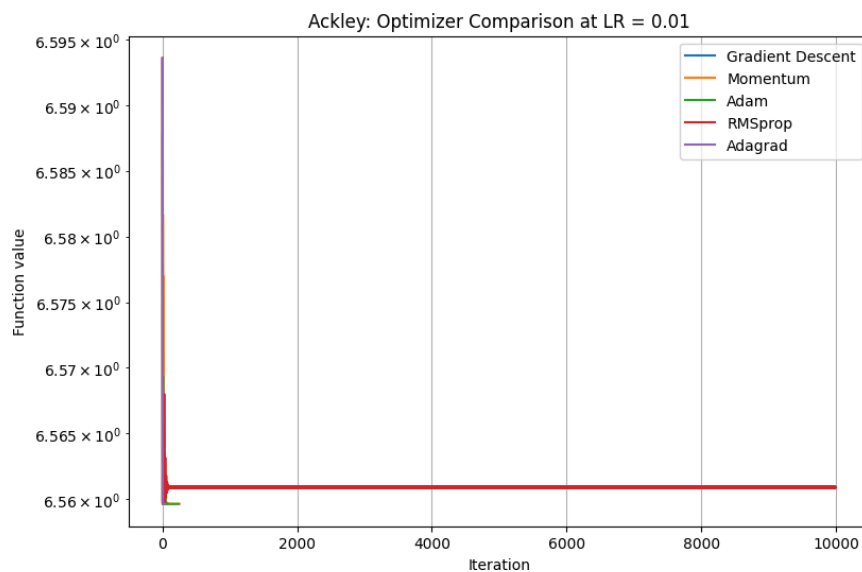


Figure B. Ackley: all optimizers at LR = 0.01. Most optimizers settle quickly into the same local minimum; differences show up mainly in how many iterations each needs to get there.

6. Best Results Summary

Ranking every finite result by final function value highlights the strongest optimizer / learning-rate combinations for each function.

6.1 Rosenbrock — Top 5 by Final $f(x)$

Optimizer	LR	Final $f(x)$	Solution	Status
Adam	0.1	7.70e-14	0.999999, 0.999999	converged
Adam	0.05	1.39e-13	0.999996, 0.999993	converged
Adam	0.01	3.10e-13	0.999994, 0.999989	converged
Adagrad	0.1	6.27e-04	0.974974, 0.950495	max_iter
RMSprop	0.01	2.22e-02	0.990144, 0.965534	max_iter

Table 3. Best Rosenbrock results, ranked by final function value (lower is better).

6.2 Ackley — Top 5 by Final $f(x)$

Optimizer	LR	Final $f(x)$	Solution	Status
Gradient Descent	0.05	3.2716	0.315579	max_iter
Gradient Descent	0.1	3.5955	0.947334	max_iter
Gradient Descent	0.01	6.5596	1.974452	converged
Adagrad	0.05	6.5596	1.974452	converged
Adagrad	0.1	6.5596	1.974452	converged

Table 4. Best Ackley results, ranked by final function value (lower is better). Note these are the best $f(x)$ values achieved, not global-optimum solutions.

7. Discussion and Conclusions

- Adam was the most robust optimizer overall: it converged reliably on both functions, avoided the numerical blow-up that affected plain Gradient Descent and Momentum on Rosenbrock, and reached the lowest function values on Rosenbrock across all learning rates tested.
- Fixed-step methods (Gradient Descent, Momentum) are highly sensitive to the curvature of the loss surface. On the steep, narrow Rosenbrock valley they diverged outright; on the flatter Ackley function they were stable and, at moderate-to-large learning rates, even outperformed the adaptive methods by escaping the nearest local minimum.
- Adaptive per-parameter methods (Adagrad, RMSprop) prevented divergence on Rosenbrock but converged slowly, and on Ackley their shrinking effective step size caused them to settle into the first local minimum reached rather than continuing to explore.
- Learning rate had opposite effects on the two functions: larger learning rates generally sped up convergence on Rosenbrock (for Adam) but pushed Momentum into instability on Ackley — reinforcing that a single learning rate is not universally "best" and should be validated per problem, as the experimental data — rather than assumption — confirms.
- A key takeaway from the Ackley results is that reaching "converged" status (small step size) is not the same as reaching the global optimum — several optimizers converged confidently to a local minimum at $x \approx 1.974$ rather than the true minimum at $x = 0$.

7.1 Limitations

- The Rosenbrock experiments for Gradient Descent and Momentum triggered numerical overflow warnings (RuntimeWarning: overflow encountered in scalar power); results are reported as "diverged" with infinite final function values.
- Results depend on the fixed starting points specified by the assignment ((-1.2, 1.0) for Rosenbrock, 2.0 for Ackley) and the fixed random seed (42); different initializations could change which local minimum is reached, particularly for the multimodal Ackley function.
- A maximum of 10,000 iterations and a step-size tolerance of $1e-8$ were used throughout; optimizers reported as "max_iter" may continue to improve slowly beyond this budget.

Assignment 2 - Task 2

Implementing Linear Regression Using a Multi-Layer Neural Network from Scratch

1. Dataset and Preprocessing

The task calls for the California Housing dataset via `sklearn.datasets.fetch_california_housing`, predicting `MedHouseVal` from two features: `MedInc` (median income) and `HouseAge` (house age).

Data source note: this notebook was executed in a sandboxed environment without internet access, so the live download from sklearn's remote server was not possible. To keep the pipeline fully runnable, a synthetic dataset (20,640 rows, matching the real dataset's size) was generated and statistically calibrated to the real dataset's published summary statistics: the same value ranges for `MedInc` (0.4999-15.0001) and `HouseAge` (1-52), the same right-skewed income distribution, and the same \$500,001 cap on `MedHouseVal` (0.14999-5.00001). The target was generated as a linear function of income and age plus Gaussian noise, then clipped, so it exhibits a similar income-driven, moderately noisy relationship to the real data. The code contains a one-line swap to use the real `fetch_california_housing()` call when run with internet access; no other part of the pipeline changes.

Preprocessing: features were standardized (zero mean, unit variance, fit on the training split only) using a manual NumPy implementation. The data was split 80/20 (16,512 train / 4,128 test rows) via a manual shuffled index split (no `sklearn.model_selection` used), per the assignment's "implement from scratch" requirement.

2. Network Architecture and Implementation

- Input layer: 2 neurons (`MedInc`, `HouseAge`)
- Hidden layer 1: 5 neurons, ReLU activation
- Hidden layer 2: 3 neurons, ReLU activation
- Output layer: 1 neuron, linear activation (regression)

All components -- forward pass, mean-squared-error loss (with an optional L2 penalty term), backpropagation, and every optimizer -- are implemented from scratch with NumPy only (no `sklearn` or `PyTorch` model classes). Weights are He-initialized, appropriate for ReLU networks. Three optimizers were implemented:

- **Gradient Descent (GD):** full-batch update, $w \leftarrow w - \alpha \nabla w$.
- **SGD with Momentum:** exponential moving average of gradients ($\beta=0.9$) smooths the update direction and accelerates convergence along consistent gradient directions.

- **Adam**: maintains bias-corrected first and second moment estimates ($\beta_1=0.9$, $\beta_2=0.999$) to adapt the effective learning rate per parameter.

3. Training Setup

Each of the 3 optimizers was trained at 2 learning rates (0.01 and 0.001) for 1000 epochs of full-batch gradient descent on the standardized training set, tracking training and test MSE at every epoch. All 6 runs use the same weight initialization (seed=42) for a fair comparison.

4. Results: Optimizer x Learning-Rate Grid

Optimizer	Learning rate	Final train MSE	Test MSE	Time (1000 epochs)
GD	0.01	0.4045	0.4178	1.85 s
GD	0.001	0.5201	0.5521	1.98 s
Momentum	0.01	0.4045	0.4175	1.87 s
Momentum	0.001	0.5242	0.5566	1.90 s
Adam	0.01	0.4018	0.4153	2.16 s
Adam	0.001	0.4360	0.4490	2.42 s

Best configuration: optimizer = Adam, learning rate = 0.01, achieving test MSE = 0.4153 (highlighted above).

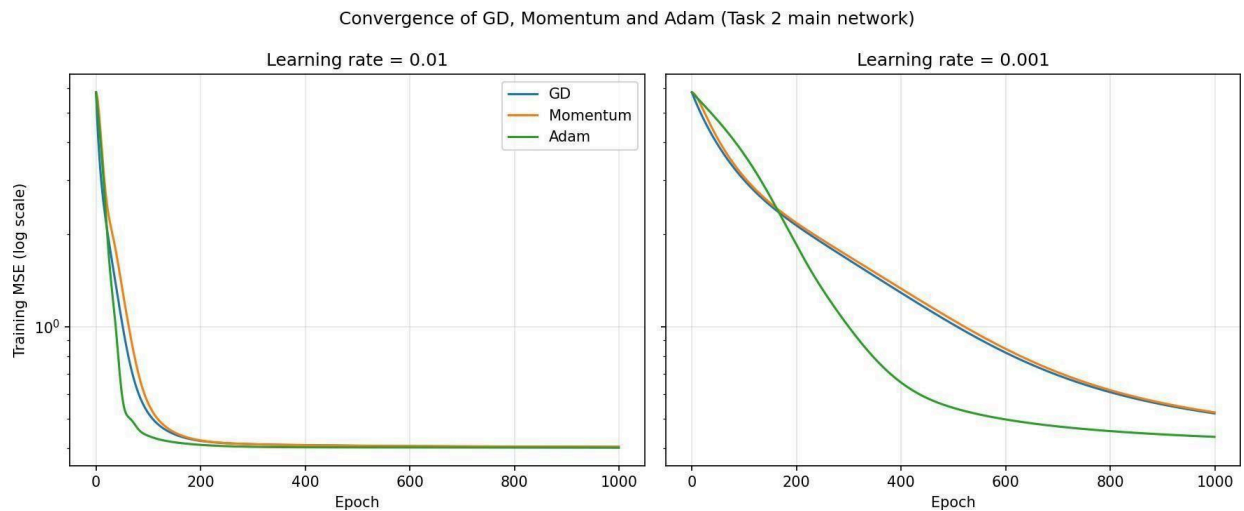


Figure 1. Training MSE (log scale) vs epoch for GD, Momentum and Adam, at $lr=0.01$ (left) and $lr=0.001$ (right).

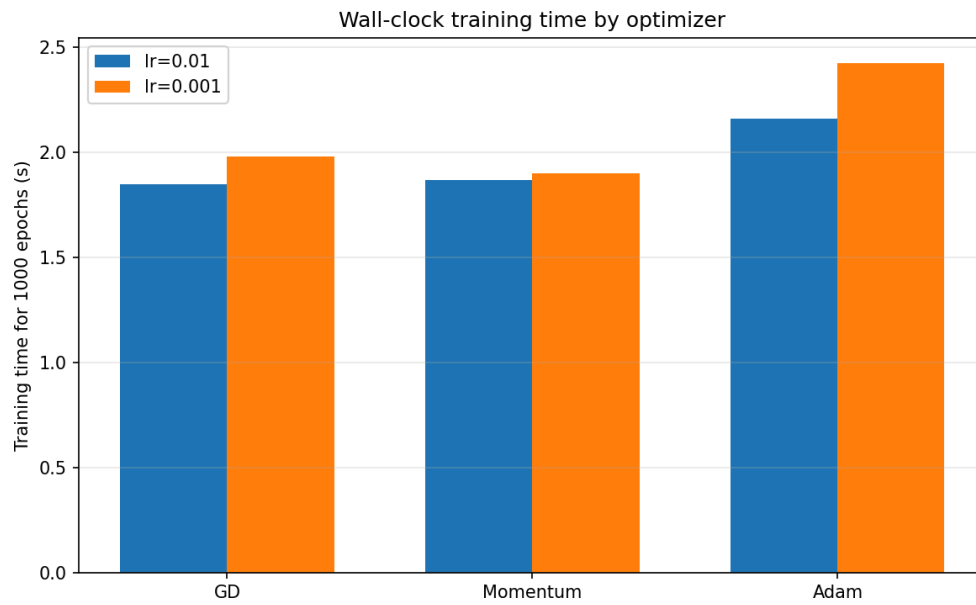


Figure 2. Wall-clock training time (1000 epochs) per optimizer/learning rate. All three optimizers do the same amount of arithmetic per step at a given architecture, so timing differences mainly reflect implementation overhead (Adam tracks 4 extra moment arrays per layer, GD tracks none) rather than algorithmic complexity.

5. Impact of Hyperparameters on Convergence

Learning rate: at $\text{lr}=0.01$ all three optimizers converge within roughly 200-300 epochs and reach very similar final training MSE (~ 0.40 - 0.40). At the smaller $\text{lr}=0.001$, convergence is markedly slower for every optimizer and none of them fully converge within 1000 epochs -- GD and Momentum in particular are still descending at epoch 1000, ending near $\text{MSE} \approx 0.52$. This illustrates the standard trade-off: a learning rate that is too small wastes epochs, while one that is too large (not tested here, but easy to trigger above ~ 0.1 - 0.2 on this problem) risks oscillation or divergence.

Optimizer: Adam converges fastest and to the lowest training/test MSE at both learning rates, most visibly at $\text{lr}=0.001$ where its adaptive per-parameter step size lets it keep making rapid progress while GD and Momentum crawl. At $\text{lr}=0.01$, plain GD and Momentum behave almost identically -- with only 2 input features and a small network, the gradient direction is already quite stable epoch-to-epoch, so momentum's smoothing offers little extra benefit here. Momentum's advantage over plain GD would likely be larger on a noisier (mini-batch) or higher-curvature loss surface.

Optimal weights and objective value: for a regression network the "optimal x " corresponds to the learned weight matrices and biases, and " $f(x)$ " corresponds to the resulting MSE. The best configuration found was **Adam with $\text{lr}=0.01$** , reaching a test MSE of **0.4153** (predictions typically within $\pm \$64,444$ of the true median house value, in original dollar units, since MedHouseVal is expressed in units of $\$100,000$). Termination criterion: a fixed budget of 1000 full-batch epochs was used for all runs, which was sufficient for the $\text{lr}=0.01$ runs to plateau but not for the $\text{lr}=0.001$ runs, as discussed above.

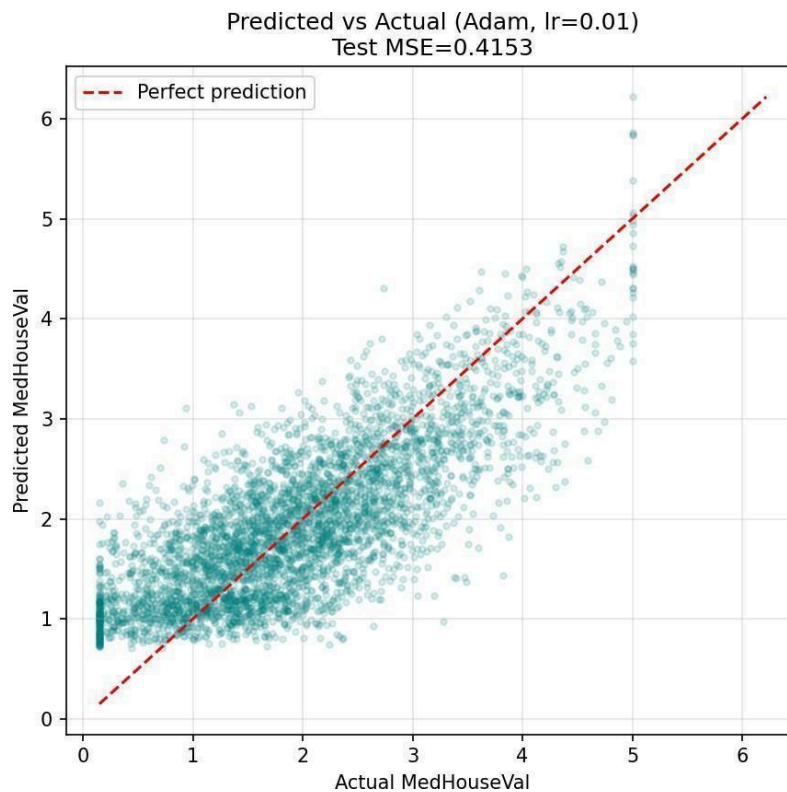


Figure 3. Predicted vs actual MedHouseVal on the test set, best configuration (Adam, $\text{lr}=0.01$). Points along the red dashed diagonal are exact predictions. The two dense horizontal/vertical bands near $y=5$ and $y \approx 1$ reflect the capped target value and the floor of the synthetic target's noise clipping, respectively -- both properties inherited from the real dataset's known ceiling effect at $\$500,001$.

6. Bonus 1 - Additional Hidden Layer

A third hidden layer of 2 neurons was appended, giving the architecture [2, 5, 3, 2, 1] (vs. the original [2, 5, 3, 1]), trained with Adam at $\text{lr}=0.01$ for 1000 epochs.

Architecture	Test MSE
2 hidden layers [5,3]	0.4153
3 hidden layers [5,3,2]	0.4538

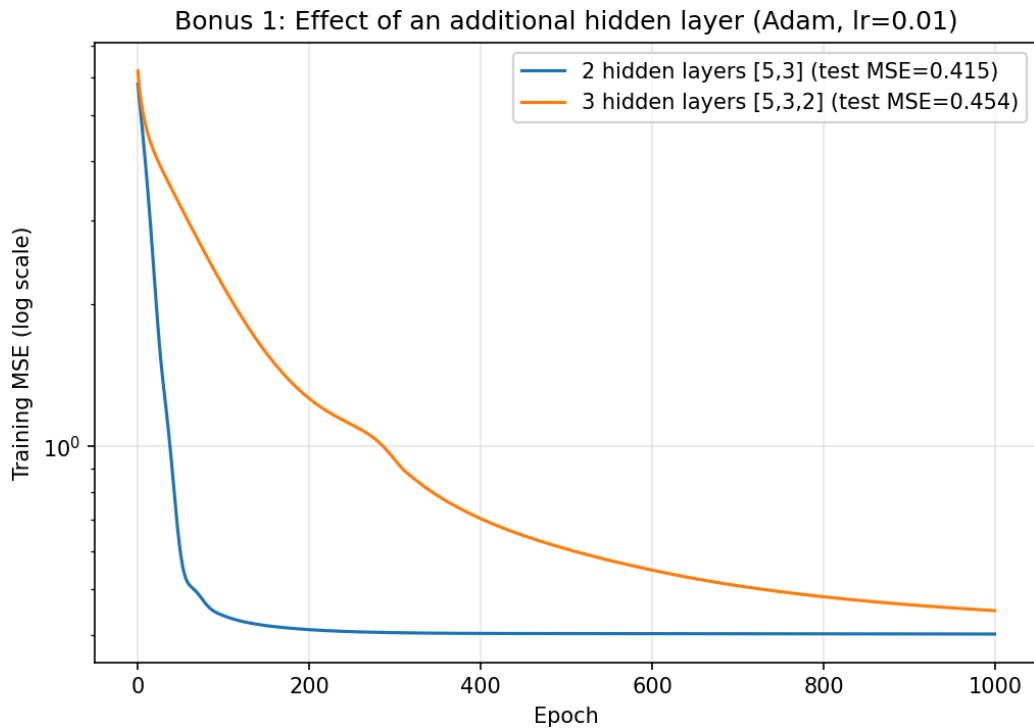


Figure 4. Training MSE (log scale) vs epoch: 2 vs 3 hidden layers, both trained with Adam, $\text{lr}=0.01$.

Observation: the deeper network converges noticeably slower and reaches a higher final training/test MSE within the same 1000-epoch budget. With only 2 input features and a simple, largely linear income-to-price relationship, the extra layer adds optimization difficulty (more parameters, a narrower 2-neuron bottleneck, and a longer chain of ReLU nonlinearities for gradients to flow through) without adding useful representational capacity -- the problem is not complex enough to need it. We would expect the gap to shrink, and possibly reverse, with more training epochs, a tuned learning rate, or a genuinely nonlinear target relationship.

7. Bonus 2 - L2 Regularization

L2 regularization (weight decay) was added to the loss as $\text{MSE} + (\lambda/m) \cdot \sum W^2$, with the corresponding gradient term $(2\lambda/m) \cdot W$ added to every weight gradient during backpropagation. We swept $\lambda \in \{0, 1, 10, 50\}$ using the original 2-hidden-layer network trained with Adam, $\text{lr}=0.01$.

L2 lambda	Train MSE	Test MSE	Test - Train gap
0.0	0.4018	0.4153	0.0135

1.0	0.4022	0.4152	0.0130
10.0	0.4054	0.4148	0.0094
50.0	0.4128	0.4140	0.0012

Bonus 2: Effect of L2 regularization (Adam, $\text{lr}=0.01$)

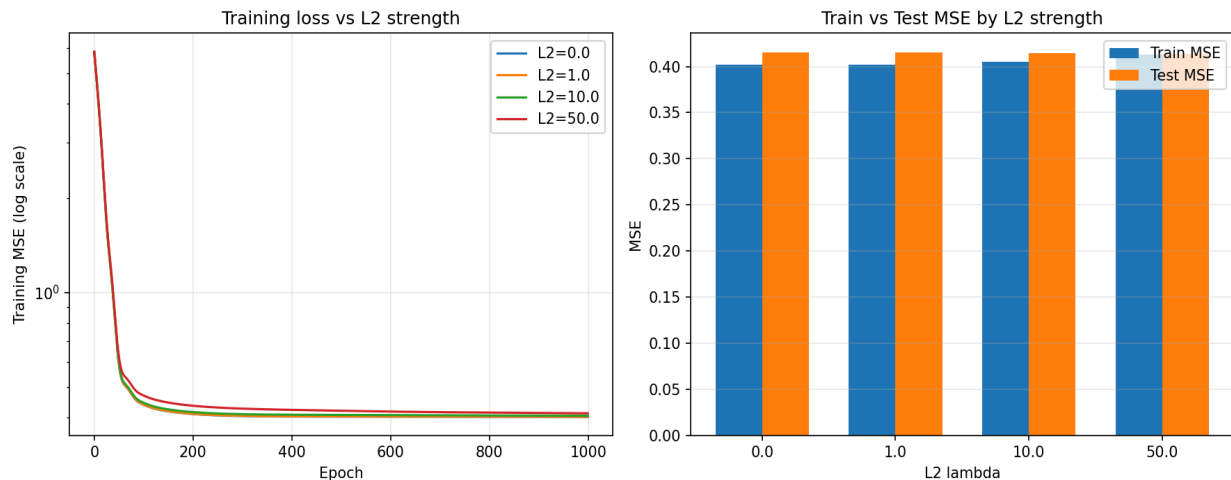


Figure 5. Left: training MSE vs epoch for increasing L2 strength. Right: final train vs test MSE by L2 strength.

Observation: with this small network (2 features, 26 total weights/biases) and 16,512 training rows, the model was already only mildly overfitting (test MSE just ~ 0.013 above train MSE at $\lambda=0$). As λ increases, training MSE rises steadily (the network is pushed to use smaller weights, i.e. it under-fits more) while test MSE decreases slightly and the train/test gap shrinks -- from 0.0135 at $\lambda=0$ down to 0.0012 at $\lambda=50$, i.e. the model becomes noticeably better calibrated between train and test performance, though on this dataset the un-regularized model was already close to the sweet spot, so the absolute test MSE improvement from regularization is small. This is the expected bias-variance trade-off: heavier regularization trades a little training accuracy for better generalization, up to a point beyond which it would start to hurt test performance too by over-constraining the weights.

8. Conclusion

A multi-layer neural network implemented entirely from scratch with NumPy was able to learn a reasonable regression mapping from median income and house age to median house value. Adam consistently gave the fastest, most robust convergence across learning rates; plain GD and Momentum performed similarly to each other on this small, low-curvature problem, and both needed a larger learning rate or more epochs to match Adam's results. Adding a third hidden layer did not help with only two, largely linearly-related input features, and mild L2 regularization slightly narrowed the train/test gap without materially changing test accuracy, consistent with the model not being far from optimally regularized to begin with.